The Long Game

Five papers on long-horizon agents — holding state, sustaining iteration, and learning from experience. A reader's guide

🤖 Long-Horizon Agents Papers · digest of 7 June 2026

June 7, 2026

Chapter 0: The Long Game

For a few years the frontier of language models was a frontier of *responses*. Could the model answer the question, write the function, summarize the document — in one shot, well. The five papers in this bundle, all posted within a single week, mark how decisively that frontier has moved. None of them is really about the quality of a single answer. Every one is about what happens when an agent has to operate over the long run: across dozens of turns, across hours of wall-clock time, across iterations of its own self-improvement. Read together, they sketch a coherent picture of where agents break in the long game, and what the field is doing about it.

Thread one: the bottleneck is state, not smarts

The two benchmark papers are diagnoses, and they independently arrive at the same uncomfortable verdict. **LongDS-Bench** shows that frontier models lose the thread over a long data-analysis session — the best one manages under 49% accuracy, with performance falling nearly 47 points from early turns to late ones. **AutoLab** shows that over hours of iterative optimization, what separates the

winners from the losers isn't raw coding ability but the discipline to keep iterating. The shared finding underneath both is that the failures are not capability failures in the usual sense. The agents in LongDS weren't running out of context; they were mis-modeling how their own state evolved. The agents in AutoLab weren't bad coders; they quit early or burned their budget flailing. Both papers explicitly rule out the obvious fix — more steps, more budget — and point instead at something structural: the ability to *hold and manage state* over a long horizon. That's the conviction to carry into the rest of the guide. Long-horizon competence is a distinct skill, and bigger models don't automatically have it.

Thread two: context is the resource you actually manage

If state is the bottleneck, context is where the fight happens — and three of these papers converge on the same engineering instinct from different directions. LongDS-Bench locates failure in state management rather than literal forgetting: the information was present, but the agent's model of "what's current" went stale. **OpenWebRL** confronts the brute version of the problem — a 30-step trajectory of screenshots overflows even a 64k-token model — and resolves it by separating visual grounding from long-term memory: keep only the last few raw observations, and let the agent's own written reasoning carry the memory of everything earlier. **JAMEL** attacks the latent version of the same problem, compressing history into vectors. The thread is that context isn't a passive buffer you fill until it's full; it's a resource you actively curate, deciding what stays raw, what gets abstracted into reasoning or principles, and what gets dropped. The naive strategy — retain everything — is both unaffordable and, several of these papers argue, unnecessary, because that's not how competent humans work either.

Thread three: experience is the engine, and it doesn't run itself

The three training-side papers all bet on the same fuel — *experience* — and all three find it requires careful handling. OpenWebRL learns from fresh attempts on the live web via online RL. **Rethinking Continual Experience Internalization** tries to bake accumulated experience permanently into the weights, and discovers that the naive loop quietly collapses: agents get *worse* across iterations unless you store experience as abstracted principles, inject it at the decision point it applies to, and learn from high-quality off-policy teachers rather than the agent's own flawed rollouts. JAMEL goes one step earlier in the pipeline and asks how an agent generates worthwhile experience at all, coupling exploration and memory so each supervises the other. The shared conviction is that experience is the path to a self-improving agent — but the shared warning is that none of it is free. Self-improvement loops can degrade silently. Memory has no labels by default. The experience you internalize can poison you if it's the wrong granularity. The papers that bet on experience are also the papers most insistent that you measure, abstract, and supervise it deliberately.

What to carry into the chapters

A few practical convictions run through everything that follows, and they're worth holding as you read. First: if you're building anything that runs more than a handful of turns, invest in an explicit, inspectable model of *current state* — don't assume a bigger context window or more reasoning steps will save you, because two of these papers specifically show they won't. Second: treat context as something you curate, keeping recent raw observations for grounding and pushing older history into compact reasoning or abstracted principles. Third: if you're running any kind of self-improvement or experience loop, measure performance across iterations, not within them, because the failure mode here is a quiet decline that looks like progress one step at a time.

The chapters ahead take each paper on its own terms — two careful diagnoses of where long-horizon agents fail, then three distinct treatments from the training side. They're ordered to move from problem to solution, but the threads above are what make them one story rather than five. The era of judging a model by its best single answer is ending. The agents that matter now are the ones that can play the long game.

Chapter 1: When the Agent Forgets What It Just Did

Paper: LongDS-Bench: On the Failure of Long-Horizon Agentic Data Analysis · arXiv 2605.30434 · **Code:** github.com/zjunlp/DataMind

Picture a data analyst three hours into a session. They've cleaned a Netflix catalog, defined a half-dozen reusable metrics, filtered the data two different ways, rolled one of those filters back when it looked wrong, and now someone asks them to compare the current scores against a baseline they computed twenty turns ago. A competent human keeps a running mental model of all of that — what's defined, what's been revised, what the "current" version of any table actually is. The question that animates this paper is uncomfortable: can an LLM agent do the same? The answer, measured carefully, is mostly no.

A benchmark built from real working sessions

The team behind LongDS-Bench wanted to test the thing that most data-analysis benchmarks quietly skip. Plenty of benchmarks hand an agent an isolated, self-contained task and reset the environment afterward. Real analysis doesn't work that way — scopes, metrics, assumptions, and intermediate results accumulate and shift across a long persistent session. So they built LongDS from real-world Kaggle notebooks: **68 tasks spanning 2,225 turns** across six domains including Geoscience, Business, and Education, with an average dependency span of **11.3 turns**. That last number is the whole point. A given request often depends on something the agent did eleven turns earlier.

What makes the benchmark sharp is that the tasks are organized around specific *state-evolution patterns*, not just length. An agent has to update a state, apply a counterfactual perturbation ("re-run this but with the duration cutoff at 100 instead of 110"), roll back to an earlier state, and compose multiple states together. Each request is deliberately constructed so the correct answer hinges on selecting the right version of an evolving analytical state — the right filter, the right definition, the right intermediate table — out of everything that came before.

The numbers are bleak, and that's the finding

Five frontier models were evaluated: GPT-5.4, Gemini-3.1-Pro, and Claude-4.6-Sonnet among them. The best of them, Gemini-3.1-Pro, reached just **48.45% average accuracy**. Every model scored below 50%. More telling than the average is the shape of the failure: accuracy **drops nearly 47 points from early turns to late turns**, and long-horizon errors — the ones specific to tracking state over many steps — account for **52% to 69% of all failures**. The agents can answer the first few questions in a session competently and then steadily lose the thread.

The most counterintuitive result is about effort. You might assume that giving the agent more steps, more room to think and re-check its work, would help. It doesn't, consistently. The authors find that **additional agent steps do not reliably improve accuracy**. Claude-4.6-Sonnet used by far the most steps (170 on average versus GPT-5.4's 69) and still didn't lead on accuracy. The bottleneck isn't interaction budget. It's maintaining a correct analytical state.

Why this should change how you think about long sessions

The pushback here is against a comfortable assumption — that long-horizon failures are basically a context-window problem, solvable by bigger context or more reasoning steps. LongDS-Bench says no: the agents in this study weren't running out of room, they were running out of *fidelity*. They lost track of which version of a thing was current. They re-used a stale definition. They failed to roll back cleanly. The errors were overwhelmingly about state management, with context-memory errors (literally forgetting earlier content) occurring comparatively rarely. The information was in the context; the agent just modeled its evolution wrong.

For anyone building agents that run for more than a handful of turns, the practical reframing is worth sitting with. If you're shipping a data-analysis agent, a coding agent, or anything that maintains state across a session, the highest-leverage place to invest is probably not a bigger context window or a longer scratchpad. It's an explicit, inspectable model of *current state* — what's defined right now, what was revised, what version of each artifact is live — that the agent reads from and writes to deliberately rather than reconstructing from raw history every turn. You could audit your own agent the way this benchmark does: feed it a sequence where turn 24 depends on a rollback of turn 21, and watch whether it grabs the right version. The cheap fix — "give it more steps" — is exactly the one the paper rules out. The harder fix — engineer the state, don't just extend the trace — is the one it points to.

That tension between scale and structure runs straight into the next paper, which asks a related question from the opposite direction: not whether agents can remember across a long session, but whether they have the *stamina* to keep iterating toward a better answer over hours of wall-clock time.

Chapter 2: Stamina Beats Brilliance

Paper: AutoLab: Can Frontier Models Solve Long-Horizon Auto Research and Engineering Tasks? · arXiv 2606.05080 · **Code:** github.com/autolabhq/autolab

Scientific and engineering progress isn't a single brilliant guess — it's a loop. You propose a change, run the experiment, measure the result, and refine, over and over, for hours. AutoLab takes that observation seriously and turns it into a benchmark that most evaluations are structurally unequipped to run: ultra-long-horizon, closed-loop optimization measured not in tokens but in **wall-clock hours**. The headline finding lands like a gut-check for anyone who equates "smart model" with "good agent." What predicts success isn't how good the agent's first attempt is. It's whether the agent keeps going.

Optimization under a clock

AutoLab is **36 expert-curated tasks** across four domains: system optimization, puzzle and challenge problems, model development, and CUDA kernel optimization. Each task starts the agent off with a baseline that is correct but deliberately suboptimal, then asks it to improve that baseline within a strict wall-clock budget — ranging from two hours for the smallest puzzles to **twelve hours** for end-to-end model post-training. The scoring is continuous, rewarding partial progress rather than pass/fail, which is what lets the benchmark see the difference between an agent that nudged the baseline a little and one that transformed it.

The scale of the evaluation is itself a data point: testing **17 state-of-the-art models** consumed **2,544 wall-clock hours and 8.60 billion tokens**. This is not a benchmark you run on a laptop over lunch. And because the tasks involve squeezing real performance out of real artifacts — faster CUDA kernels, better-tuned models — the authors had to defend against agents that try to win by cheating the scorer. They ran a dedicated adversarial agent prompted to find reward hacks and verifier exploits, and patched the holes it found. When you reward an agent for a measured number going up, some fraction of models will try to make the number go up without doing the work.

The dominant predictor is persistence

Here's the result that reframes everything. Across the 36 tasks, **final performance correlates far more strongly with persistence than with one-shot solution quality**. The agents that win are the ones that repeatedly benchmark, edit, and fold empirical feedback back into the next attempt. The agents that lose fail in one of two characteristic ways: some **terminate prematurely**, declaring victory or giving up after minimal exploration, while others **exhaust their entire budget** flailing without ever producing a valid final solution. Both failure modes are, notably, "unrelated to raw coding ability." A model can be a strong coder and still be a poor long-horizon agent because it lacks the disposition to keep iterating.

One model stood out. **claude-opus-4.6** led all four categories and the overall ranking, reaching an Avg@3 of 0.68 — a substantial margin over the field. Most other frontier models, including several proprietary ones, struggled to sustain the loop. The gap between the best and the rest is not primarily a gap in intelligence; it's a gap in *time awareness and iterative discipline*.

What "persistence" means for the agents you build

The implication cuts against a habit that's easy to fall into when prompting agents: optimizing for a great first answer. AutoLab suggests that for any task that plays out over a long horizon — anything where the agent has hours and a measurable objective — the first answer barely matters. What matters is the agent's behavior in the middle of the run: does it keep benchmarking its current solution, does it actually read the empirical feedback and act on it, and does it manage its time budget instead of quitting early or burning the whole thing on a dead end.

If you're building or evaluating this kind of agent, there are concrete things to borrow. Give the agent explicit awareness of its remaining wall-clock or compute budget, and instrument whether it's allocating that budget across exploration and refinement rather than spending it all upfront or bailing halfway. Watch your trajectories for the two failure modes AutoLab names — premature termination and budget exhaustion — because they're diagnosable and they're probably costing you more than any single bad decision. And if you reward a measured outcome, assume some models will try to game it; the adversarial-agent-as-red-team pattern is a cheap way to find the exploits before your benchmark blesses a cheater. Most usefully: when you compare models for long-horizon work, the best one-shot coder on a static benchmark may not be the most persistent iterator, and persistence is what this regime rewards.

Both this paper and the last one are diagnoses — careful accounts of where long-horizon agents break. The next three papers shift from diagnosis to treatment, each attacking the problem from the training side. The first takes on the visual web, asking whether you can train a small agent to navigate live, messy, real-world websites by letting it learn directly from its own attempts.

Chapter 3: Training a Web Agent on the Live Web

Paper: OpenWebRL: Demystifying Online Multi-turn Reinforcement Learning for Visual Web Agents · arXiv 2606.02031 · **Code:** github.com/OpenWebRL/OpenWebRL

The strongest visual web agents are proprietary, and the open ones have a structural dependency that caps how good they can get: they're trained by imitation, on large hand-curated collections of web trajectories. Those demonstrations are expensive to collect, and a static dataset can only ever cover a frozen slice of an open web that's constantly changing. OpenWebRL's wager is that you can break that ceiling by letting the agent learn from its own attempts — online reinforcement learning, run directly against live websites — and that you don't need a giant model to do it. Their **OpenWebRL-4B** model, a 4-billion-parameter agent, reaches **67.0% success on Online-Mind2Web and 64.0% on DeepShop**, beating open agents of similar or larger size and staying competitive with proprietary systems like OpenAI CUA and Gemini CUA.

A recipe, not a black box

The paper's framing is deliberate: rather than treat online RL as a black-box trick, OpenWebRL lays out the full pipeline so others can reproduce it. There are three load-bearing pieces. First, a **supervised warm start** — a light layer of imitation learning on just **0.4K trajectories** to get the policy into a productive region before RL begins. Second, the RL loop itself, run on live browsers with multimodal context management. Third, a **multi-turn GRPO** objective with trajectory-level judging, where success on a whole episode is scored and that signal is propagated back to the individual actions. The entire RL stage uses only **2.2K open-ended training tasks**, which is strikingly little.

A few design decisions are worth pulling out because they generalize beyond web navigation. The agent uses a generic ReAct-style tool-calling harness rather than a web-specific scaffold, so the same paradigm extends to any tool-using domain. The live-web infrastructure is built to be fault-tolerant — navigation retries, timeout handling, structured failure attribution — precisely so that a website blocking the agent gets cleanly separated from the agent making a bad decision. On a live web full of pop-ups, redirects, and bot detection, that separation between environment noise and model behavior is what makes the training signal trustworthy at all.

Solving the screenshot problem

The most transferable idea in the paper is how OpenWebRL handles context. A visual web agent generates a full-page screenshot at every step, and a 30-step trajectory of screenshots can blow past the context budget even for a 64k-token model. The naive fix — keep every screenshot — is

both unaffordable and, the authors argue, unnecessary. Humans don't re-inspect every previous browser state; they rely on recent visuals plus a memory of what they've already done. OpenWebRL adopts the same split: **separate visual grounding from long-term memory.** Only the most recent K screenshots are retained as images, while the agent's own reasoning traces serve as a compact textual memory of everything earlier — capturing prior actions, outcomes, and progress without dragging the pixels along.

This is the same long-horizon state problem the first two chapters circled, met with an engineering answer. The ablations confirm it matters: how you manage that context measurably moves performance, and the warm start does more than just speed up RL — it provides a foundation the RL phase builds on rather than replaces. The team also distilled an **8B judge model** to score trajectory success, reaching nearly 90% alignment with a GPT-4.1 judge, so the whole pipeline can run without a proprietary model in the evaluation loop.

What you can lift from this

The headline lesson is that online RL on live environments is a viable, reproducible path to a capable agent — and you don't need a frontier-scale model or a mountain of demonstrations to walk it. A 4B model, 400 warm-start trajectories, and 2,200 RL tasks got within reach of proprietary systems. If you've been assuming a good web agent requires either a huge model or a huge imitation dataset, this is a direct counterexample.

The most immediately borrowable piece is the context strategy, and it isn't specific to web agents. Any long-horizon multimodal agent — anything that accumulates screenshots, images, or large observations over many steps — faces the same explosion. The fix is the same: keep the last few raw observations for grounding, and let the agent's own written reasoning carry the long-term memory of what happened earlier. You can apply that pattern today without touching RL at all. And if you do want to train with RL, two practical notes carry over: warm-start your policy so it isn't exploring from scratch, and invest in fault-tolerant infrastructure that distinguishes environment failures from agent failures, because a noisy reward signal is worse than a small one.

OpenWebRL learns from fresh attempts each time, but it doesn't try to permanently absorb what it learned into the model's weights. The next paper takes on exactly that harder problem — how an agent can internalize its accumulated experience into lasting, reusable capability — and finds that the obvious approaches quietly fall apart.

Chapter 4: Why Self-Improving Agents Quietly Get Worse

Paper: Rethinking Continual Experience Internalization for Self-Evolving LLM Agents · arXiv 2606.04703 · **Code:** github.com/RUCBM/ExpInternalization

The dream of a self-evolving agent is seductive: it acts, it learns from what it did, it bakes that lesson into its own weights, and the next time around it's a little better. Repeat across many iterations and the agent compounds — getting steadily smarter at its own job. This paper sets out to test whether that compounding actually happens, and the finding is sobering. Under multi-iteration experience learning, the standard methods don't compound at all. They suffer **progressive capability collapse** — the agent gets *worse* across iterations, not better. The graph that opens the paper shows success rates on web-reasoning tasks declining steadily from iteration one to iteration three.

Anatomy of a collapse

The mechanism at the center of all this is *experience internalization*: converting the contextual experience an agent picks up during interaction into reusable parametric capability, so it doesn't have to keep that experience in its context window forever. In-context learning is the most direct way to use experience, but it's bounded by context capacity and prone to collapse as the experience pool grows. Internalization promises to escape that by writing the lessons into the weights. The trouble is that most prior work only studied a *single* round of transfer. Run it iteratively — the way a genuinely self-evolving agent would — and the wheels come off.

The authors dissect the failure along three dimensions, and each one yields a practical design rule.

The first is **experience granularity** — what, exactly, gets internalized. You can store *instance-level* experience (the specific trajectory: "in this case I did X then Y") or *principle-level* experience (the abstracted strategy: "when you hit this kind of state, prefer this kind of move"). They find principle-level experience is far more durable. It abstracts the transferable strategy away from trajectory-specific details, so internalizing it doesn't drag along brittle, overfit specifics that poison later iterations.

The second is the **injection pattern** — when the experience is presented during training. *Global injection* hands the model all the relevant experience up front; *step-wise injection* aligns each piece of experience with the intermediate decision state where it actually applies. Step-wise wins, and the reason matters for long-horizon work specifically: aligning experience with the decision point it informs is exactly what tool-use tasks need, where the relevant lesson depends on where you are in the trajectory.

The third is the **internalization regime** — the training signal itself. The intuitive choice is *on-policy* context-distillation, where the student learns from its own rollouts. But on-policy learning is inherently limited to local corrections on the flawed states the student itself stumbles into — and those flawed states are the disease, not the cure. **Off-policy** context-distillation, learning from high-quality teacher trajectories instead, provides a substantially more stable signal because it isn't anchored to the student's own mistakes.

So what, and what to do about it

The broader lesson is a warning shot for anyone running an agent in a self-improvement loop. The assumption that "more iterations of learning from experience" monotonically improves an agent is wrong by default — left unchecked, the loop can degrade the very capability it's meant to grow. Worse, the collapse is quiet: each individual iteration looks like progress, and the decline only shows up when you measure across the full sequence. If you're not explicitly tracking performance iteration over iteration, you might be shipping an agent that's getting worse and not know it.

The three findings translate cleanly into things to try. When you capture experience for an agent to learn from, store it as **abstracted principles, not raw episodes** — the durable signal is the strategy, not the transcript. Inject experience **at the decision point it applies to**, not in a single upfront dump, especially for multi-step tool use where the right lesson is state-dependent. And when you choose a training signal, prefer **high-quality off-policy teacher data over the agent's own on-policy rollouts**, because learning to correct your own mistakes locally is a far less stable foundation than learning from someone who did it right. Even with all three in place the authors note degradation can still creep in — which is the real takeaway: continual self-improvement is not free, and it demands deliberate engineering plus per-iteration measurement, not a hopeful loop you set running and walk away from.

The final paper shares this paper's conviction that experience is the engine of a capable agent — but it goes after a different bottleneck. Instead of asking how to internalize experience that already exists, it asks how an agent can *generate* worthwhile experience in the first place, by learning to explore and to remember at the same time.

Chapter 5: Memory and Curiosity, Trained Together

Paper: Joint Agent Memory and Exploration Learning via Novelty Signals · arXiv 2606.01528 · **Code:** github.com/MobileLLM/JAMEL

Exploration is the capability that lets an agent operate in an open-ended environment where rewards are sparse or absent — poking at an unfamiliar app, trying things, discovering what's possible. Current LLM agents are conspicuously bad at it. Even when an agent stumbles onto unexpected but useful information mid-task, it usually fails to act on it, and the authors note this is a deficit baked in by training, not something you can prompt your way out of. JAMEL's insight is that you can't fix exploration in isolation, because exploration and memory are two halves of the same problem — and the clever move is to train them together, supervised by a signal that costs nothing to collect.

The mutually dependent loop

The core observation is that **memory and exploration form a mutually dependent loop**. To explore well, an agent needs memory: in a partially observable environment, it can't decide whether an action is worth trying unless it knows which states and behaviors it has already seen. But memory has a notoriously hard training problem. You could just retain the full interaction history, but that gets ruinously expensive as trajectories grow long. The efficient alternative is *latent memory* — compressing history into a compact vector — but latent memory lacks reliable step-level supervision. Nobody can hand-label what each compressed memory vector "should" encode, because the vectors have no human-readable semantics.

JAMEL closes the loop by letting exploration supply the missing supervision. The agent is given a **novelty reward**: it earns reward only when it reaches behavior its own history hasn't covered. Maximizing that reward forces the memory module to actually encode and use what's already been tried — because the only way to keep finding novelty is to remember what isn't novel anymore. Exploration becomes the teacher that tells memory what's worth remembering, and memory becomes the faculty that makes continued exploration possible. Each makes the other useful.

A free supervision signal

The elegant part is where the novelty reward comes from. In the GUI domain, **code coverage** is a natural, deterministic proxy for novelty. Instrument any application to report which code paths execute, and you get an annotation-free signal: novelty is awarded when the agent triggers a code path it hasn't hit before. No human labeling, no reward model, no curated demonstrations — just the application telling you, for free, whether the agent did something genuinely new.

This setup produces a curriculum for free, too. The coverage baseline persists across episodes and isn't reset, so as the easy, shallow interactions get exhausted, only **deeper multi-step sequences continue to yield novelty reward**. The reward naturally gets sparser as the session advances, pushing the policy and memory to improve together toward harder, longer behaviors — without anyone designing a curriculum by hand. Empirically, JAMEL generalizes to unseen environments, outperforming open-weight memory baselines on ten held-out apps and rivaling the exploration depth of a closed-source model while consuming fewer tokens.

What this opens up for you

The broad implication is that the hardest part of training latent memory — the absence of a supervision signal — can be dissolved by coupling it to a behavioral objective the environment can score on its own. You don't need labels for what memory should hold if you have a reward that's only achievable by holding the right things. That reframing is more general than GUIs: any domain where you can cheaply, deterministically measure "has the agent done something new?" can in principle supply the same annotation-free supervision.

Concretely, the pattern worth borrowing is **deterministic novelty as a free training signal**. Code coverage is the obvious instance for software agents, but the recipe is "find a persistent, automatable measure of novel behavior in your environment, reward the agent for triggering it, and let that reward both drive exploration and supervise what memory encodes." The persistent-baseline trick is worth stealing on its own: by not resetting the coverage tally between episodes, you manufacture an automatic difficulty ramp that keeps pushing the agent toward deeper behavior with zero curriculum design. And if you've been treating agent memory and agent exploration as separate engineering problems, JAMEL's argument is that you'll do better treating them as one — because each is the thing that makes the other trainable.

Across these five papers, a single conviction keeps surfacing: the frontier of agent capability is no longer about the quality of a single response. It's about everything that happens over the long run — holding state, sustaining iteration, managing context, internalizing experience, and generating new experience worth learning from. The synthesis chapter that opens this guide draws those threads together.